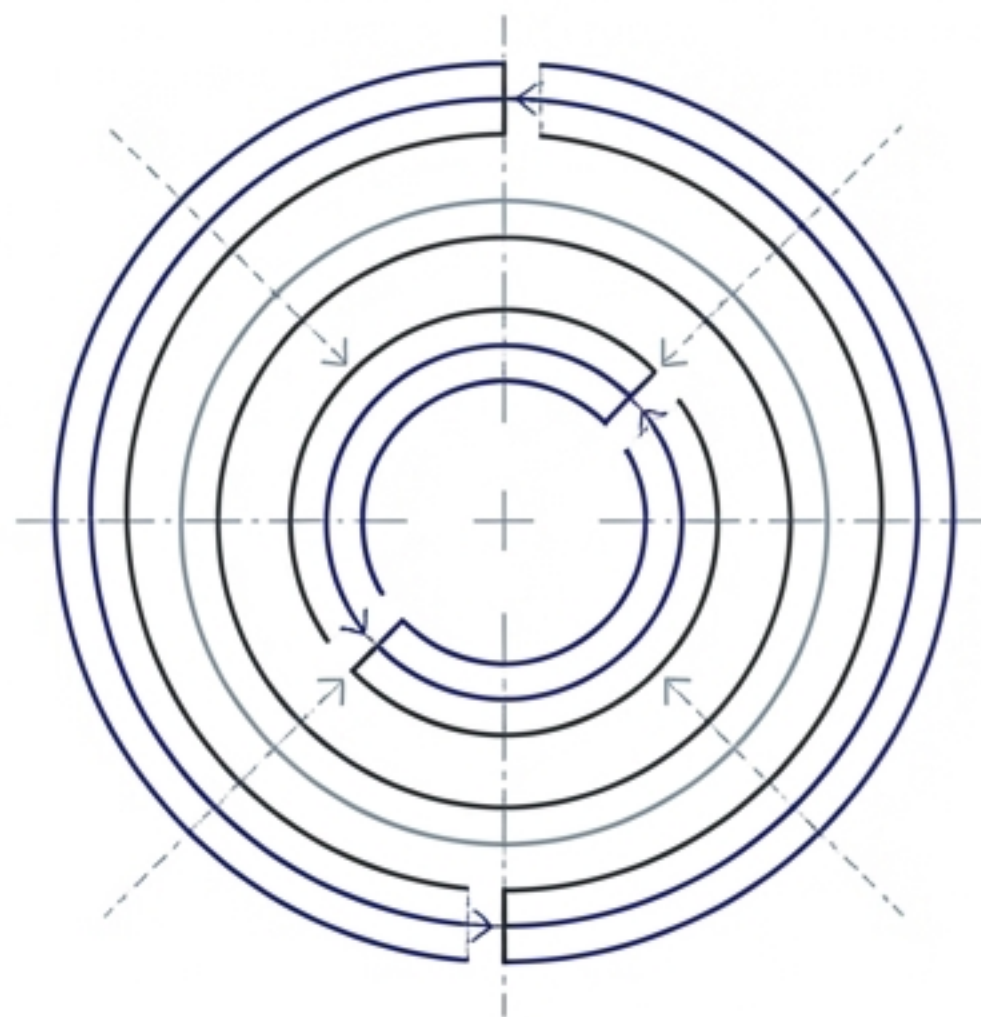


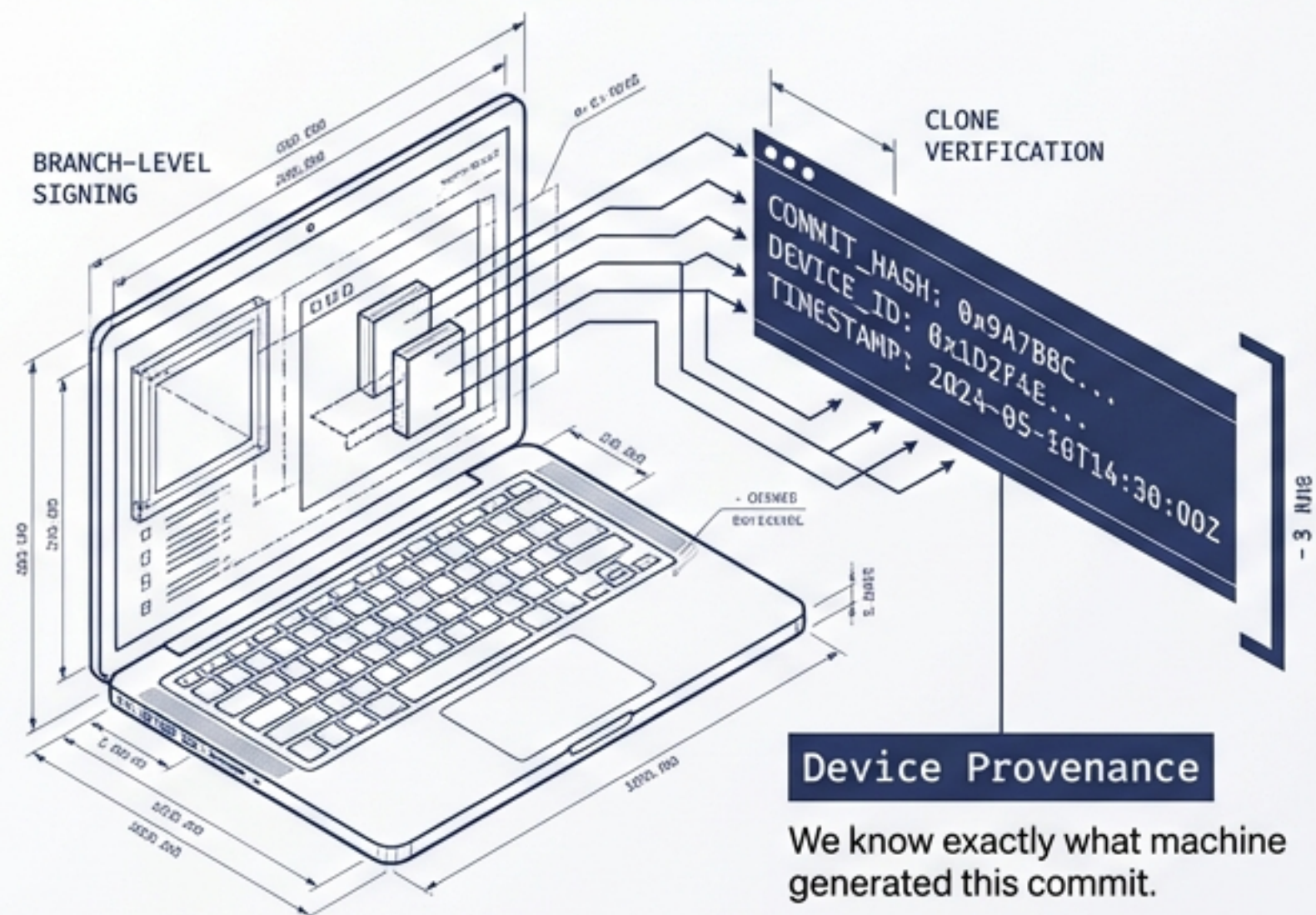


Beyond Device Provenance

Extending branch-level signing to full human identity and multi-party trust within the sg-vault ecosystem.

Audit trails and data rooms demand human attribution

The current architecture mandates branch-level signing. This proves a commit came from a specific clone, but cannot prove person identity. We need to bridge the gap between device and human without compromising the system's foundational security rule.

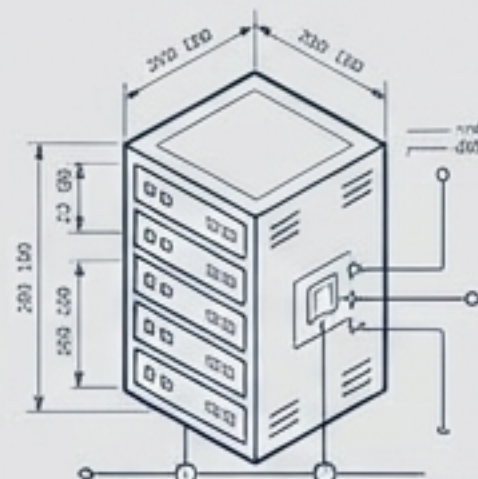


The Golden Rule: Only public keys ever enter the vault. User private keys are managed entirely outside.

The four cryptographic keys powering the vault

vault_key

The baseline secret for anonymous operations.



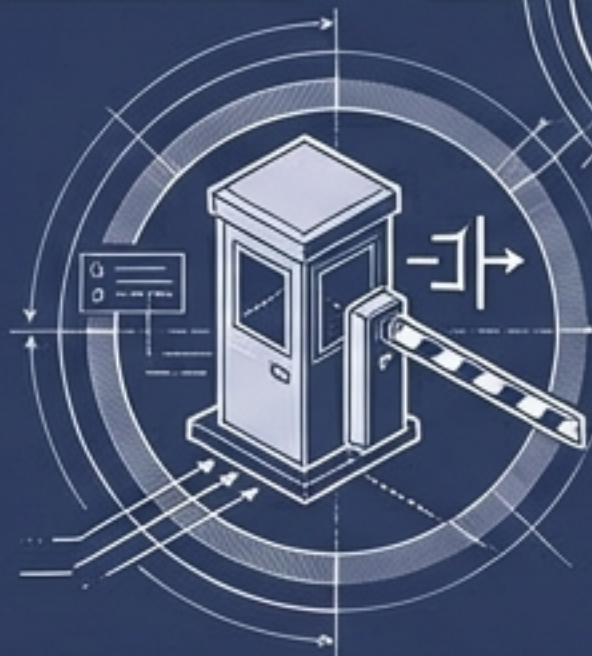
branch_key

The device identity.
Auto-generated on clone.
Mandated by current specs.



named_branch_key

The gatekeeper. Authorizes
merge signatures into main
branches.



user_key

The human identity.
Configured via pki keygen. This
is the missing link connecting
commits directly to people.



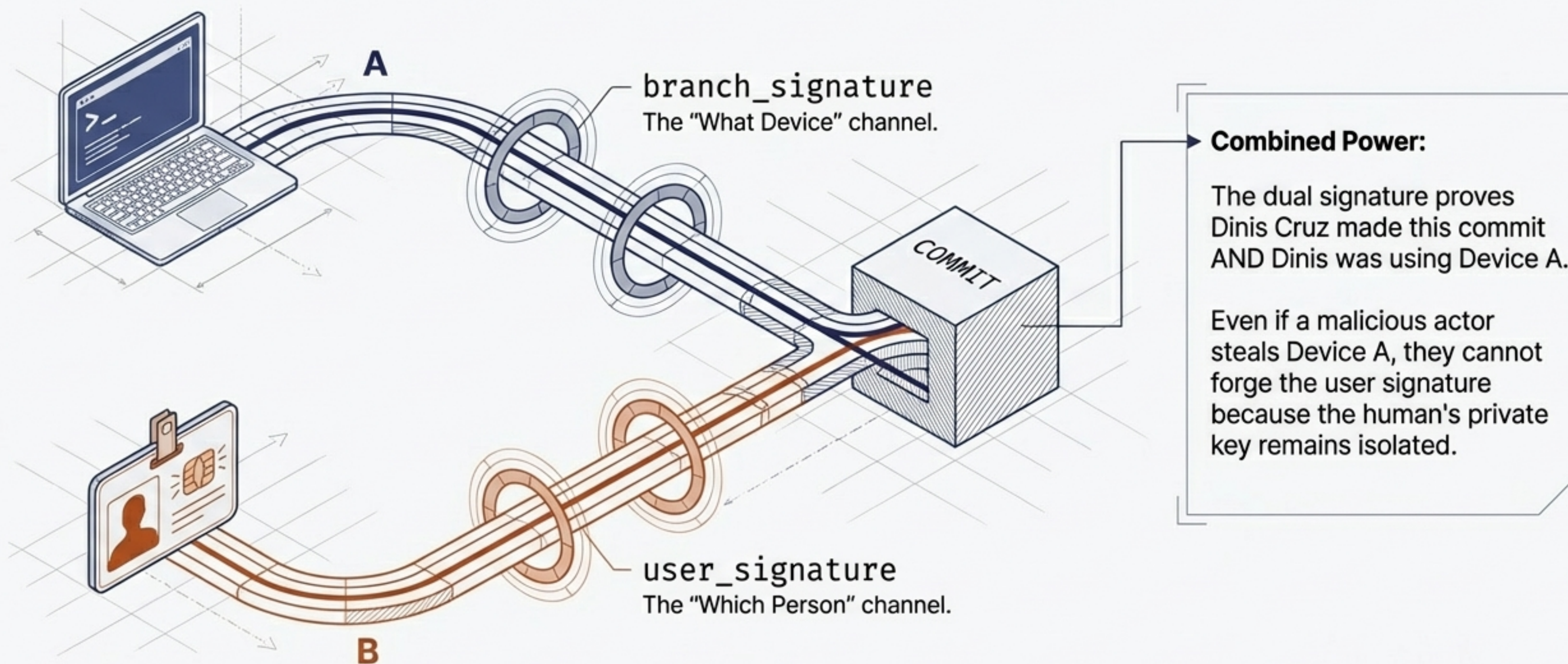
The Provenance Spectrum: Scaling trust to match compliance

	Mode 1 (Anonymous)	Mode 2 (Device)	Mode 3 (Author)	Mode 4 (Countersigned)
Keys in Play	Vault Key	+ Branch Keys	+ User Keys	+ External Keys
Tamper-Proof Chain	✗	✓	✓	✓
Proves Device	✗	✓	✓	✓
Proves Human Identity	✗	✗	✓	✓
Third-Party Auditable	✗	✗	✗	✓
Ideal Target	Scratch vaults	AI agents / Teams	Data rooms	Regulated finance

Takeaway: Modes are additive. More keys equal stronger provenance guarantees.

Dual-channel signatures provide unforgeable context

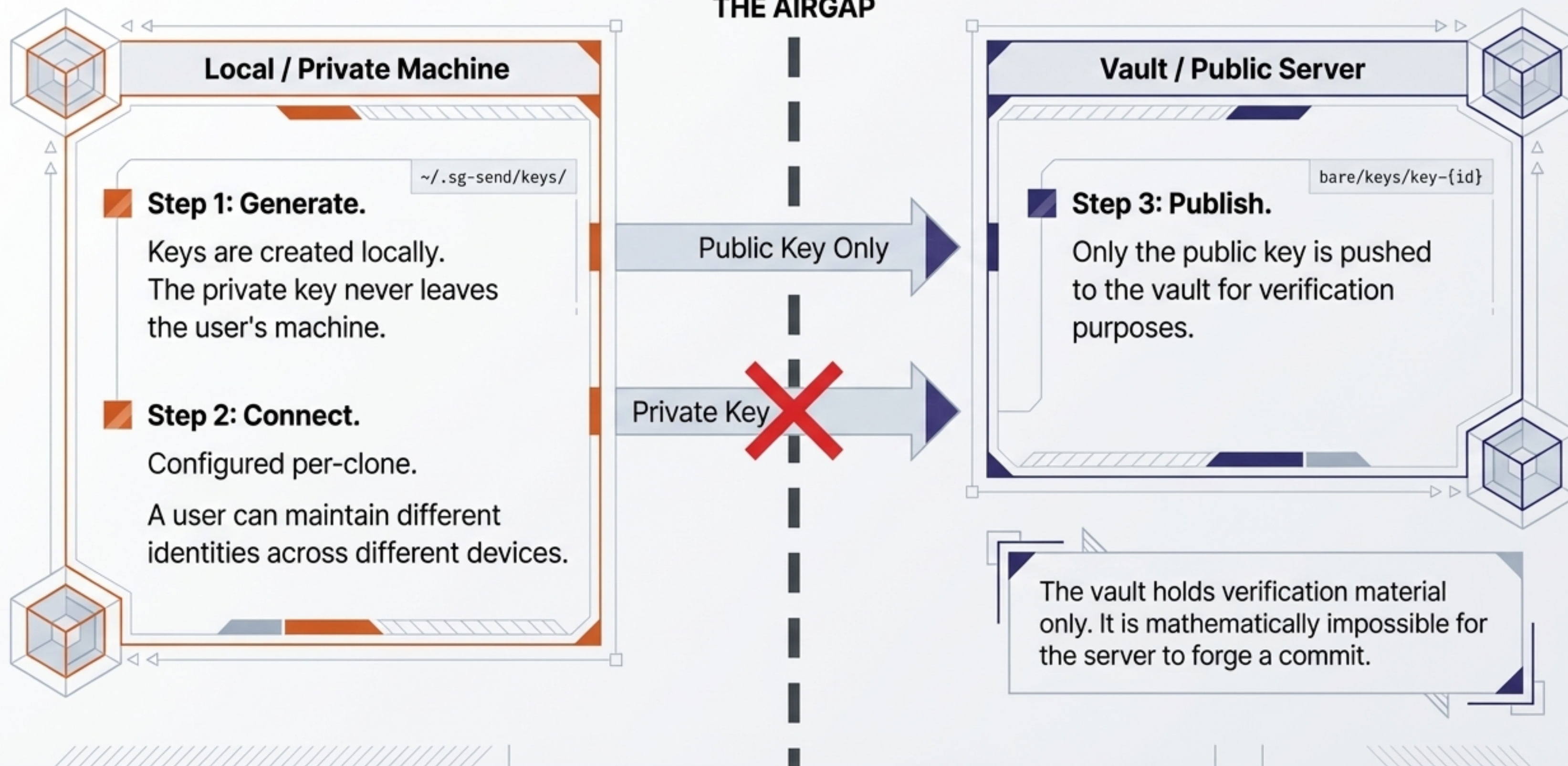
Branch and User keys are complementary signals. Neither alone provides full context.



The Trust Barrier keeps private keys permanently isolated

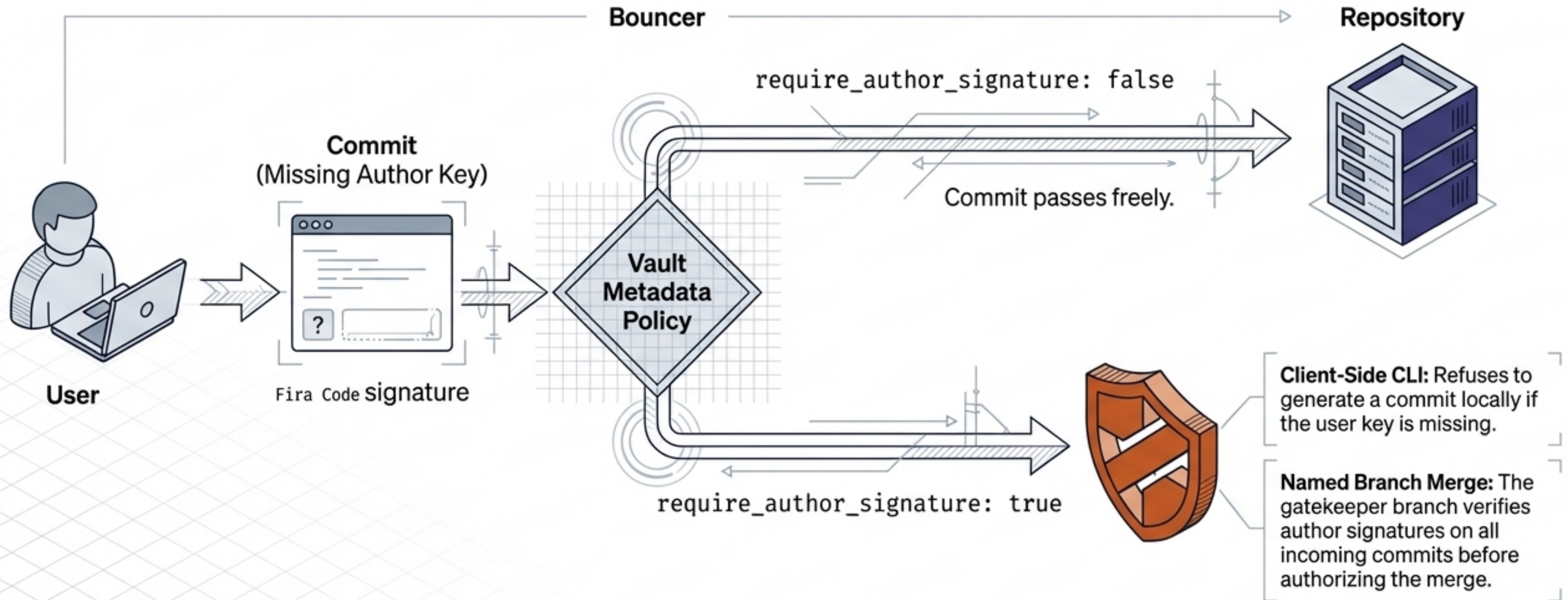
Helvetica Now Display

THE AIRGAP



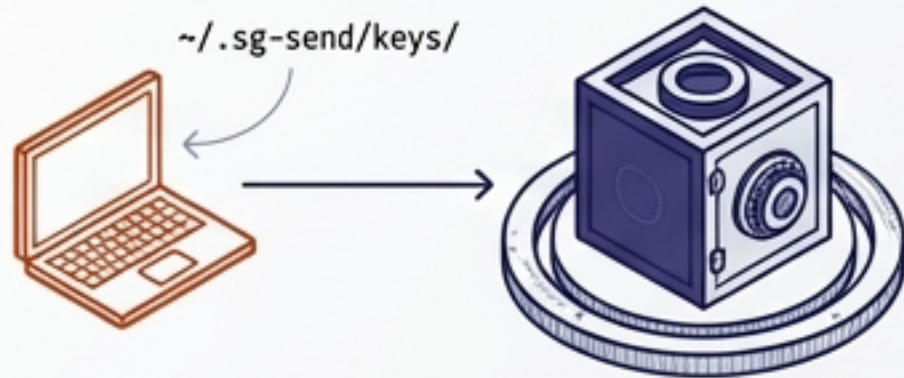
Policy enforcement is strictly client and merge-side

Because the server is zero-knowledge, compliance is enforced at the edges. Vaults declare their minimum provenance level via metadata.



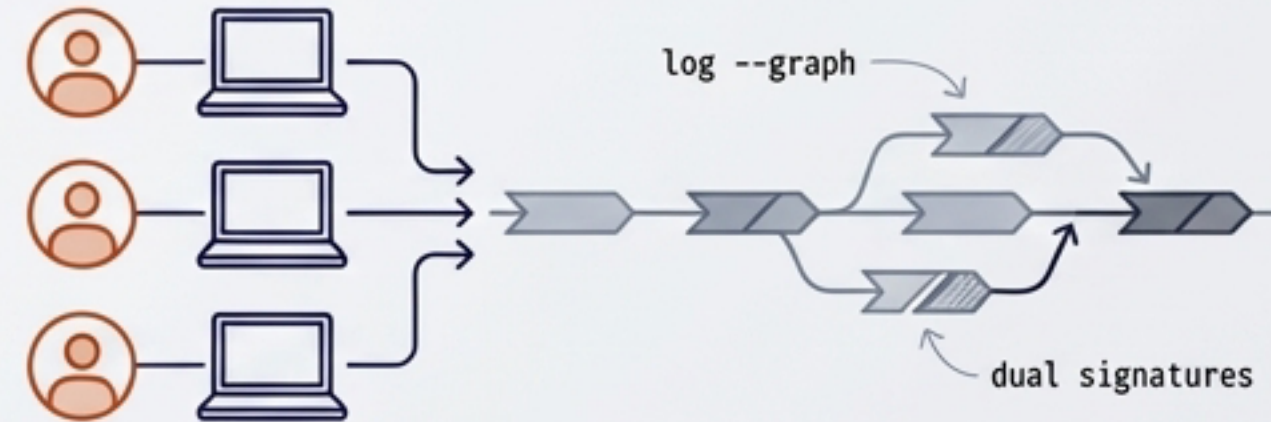
Adapting the PKI spectrum to real-world workflows

Solo Developer (Mode 2)



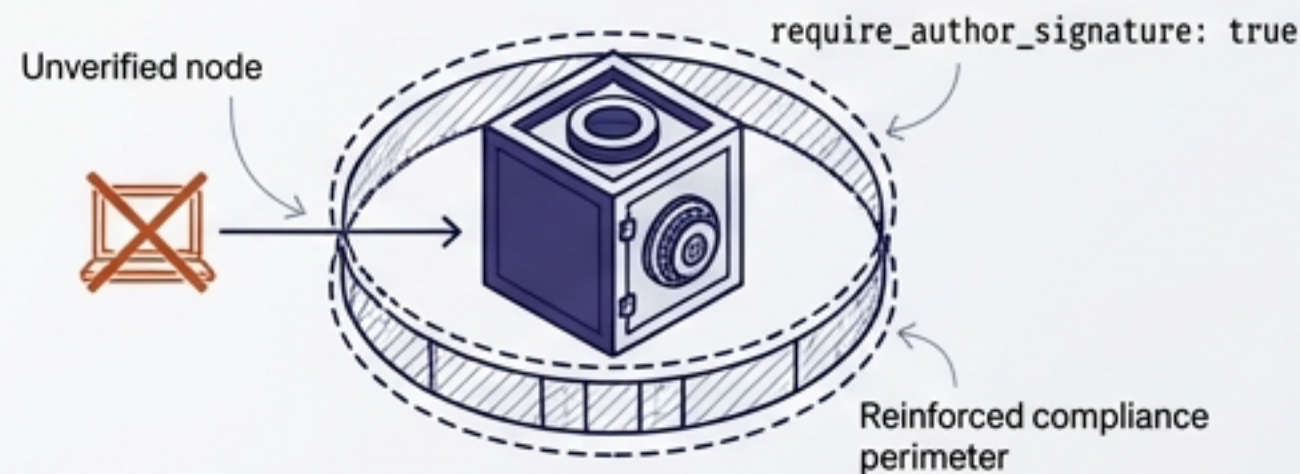
Default behavior. Auto-generated branch keys.
Zero friction. No user key required.

Team Identity (Mode 3)



Optional user keys. ~~XXXXXXXXXX~~ command `'log --graph'` reveals dual signatures, allowing team-wide attribution.

Regulated Data Room (Mode 3 Enforced)



Strict policy. Commits are systematically rejected without cryptographic proof of human authorship.

Cross-Vault Identity



Global verification. A user publishes their public key to the SG/Send PKI registry, carrying their verified identity across any vault.

metadata: {
 environment: "production-agent",
 instance_id: "a7b9f1"
}

Agents lack personal user keys. Instead, branch metadata identifies the agent's environment, while the branch key proves the same instance authored a chain of commits.

[COMMIT 01] [COMMIT 02] [COMMIT 03]

Persistent Agent Instance

The diagram illustrates a security process for injecting payloads into a vault repository. On the left, a cardboard box labeled "[CHANGE PACK PAYLOAD]" is shown with a padlock icon. A tag attached to it contains the text "creator_key: 'ck_9d8e7f'". Below this, a flowchart shows a "[EXTERNAL SYSTEM]" (represented by a key icon) sending a payload through a "CLONE PATHWAY" (indicated by a crossed-out arrow) to a "[VAULT REPOSITORY]" (represented by a safe icon). A thick orange arrow labeled "Direct Payload Injection" bypasses the clone pathway and points directly into the vault repository. A label "Verified open drain" is positioned near the vault repository.

creator_key: "ck_9d8e7f"

[CHANGE PACK PAYLOAD] [CHANGE PACK PAYLOAD]

[EXTERNAL SYSTEM]

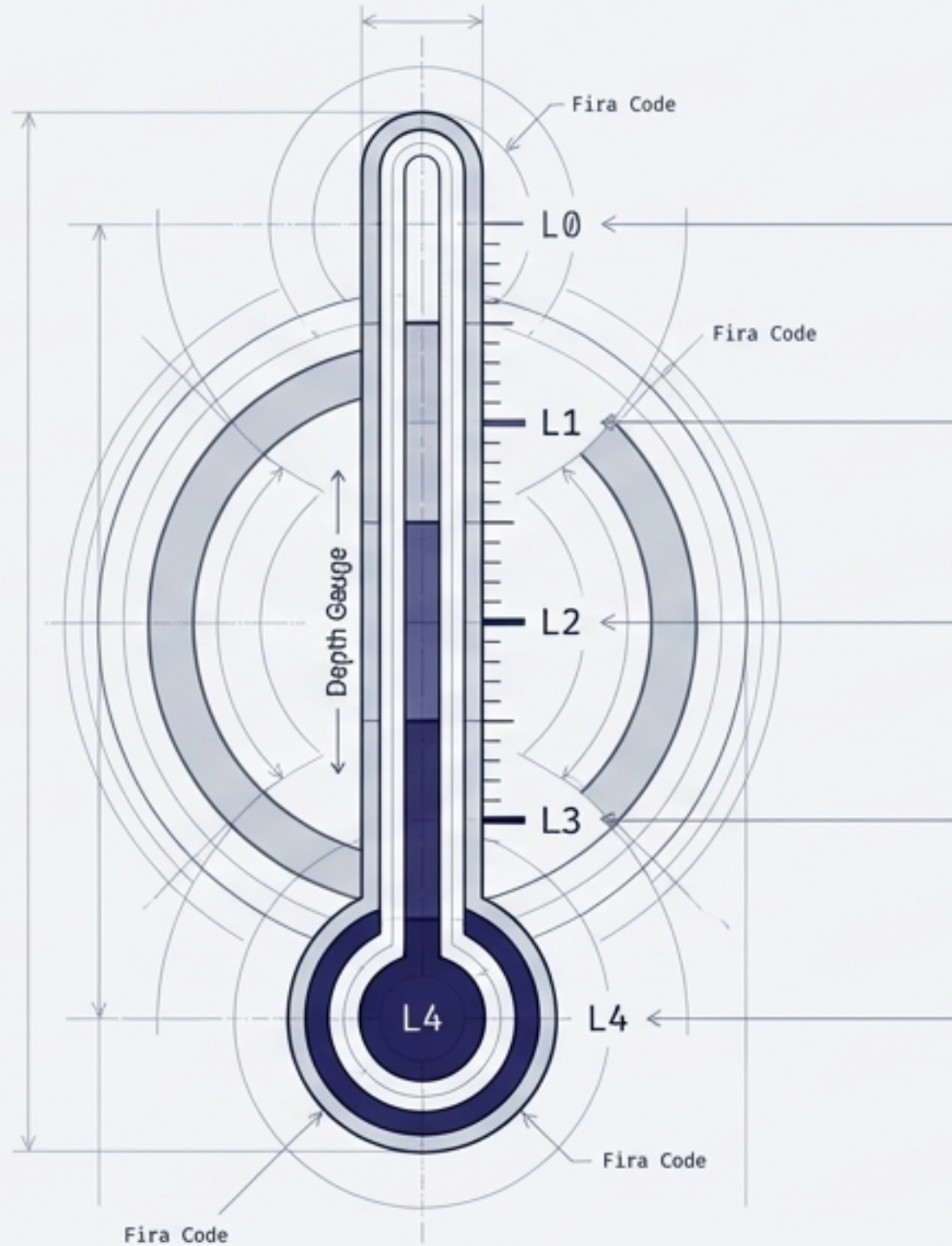
CLONE PATHWAY

Direct Payload Injection

Verified open drain

[VAULT REPOSITORY]

Signature verification depth is controlled by the client



L0: Disabled

Skips checks for raw speed.

L1: Branch Only

Validates device identity and immediate commit integrity.

L2: Author Identity

Validates the human signature against the public key registry.

L3: Chain of Trust

Deep verification of the entire commit history graph.

L4: External Attestation

(Future) Validates countersignatures from auditors or notaries.

Takeaway: The server never verifies. Clients perform all cryptographic checks to maintain zero-knowledge infrastructure.

The cryptographic reality of immutable provenance

To prevent migration pain, the v0.13.30 schema natively reserves space for Mode 3 and 4 fields, even when operating at Mode 2.

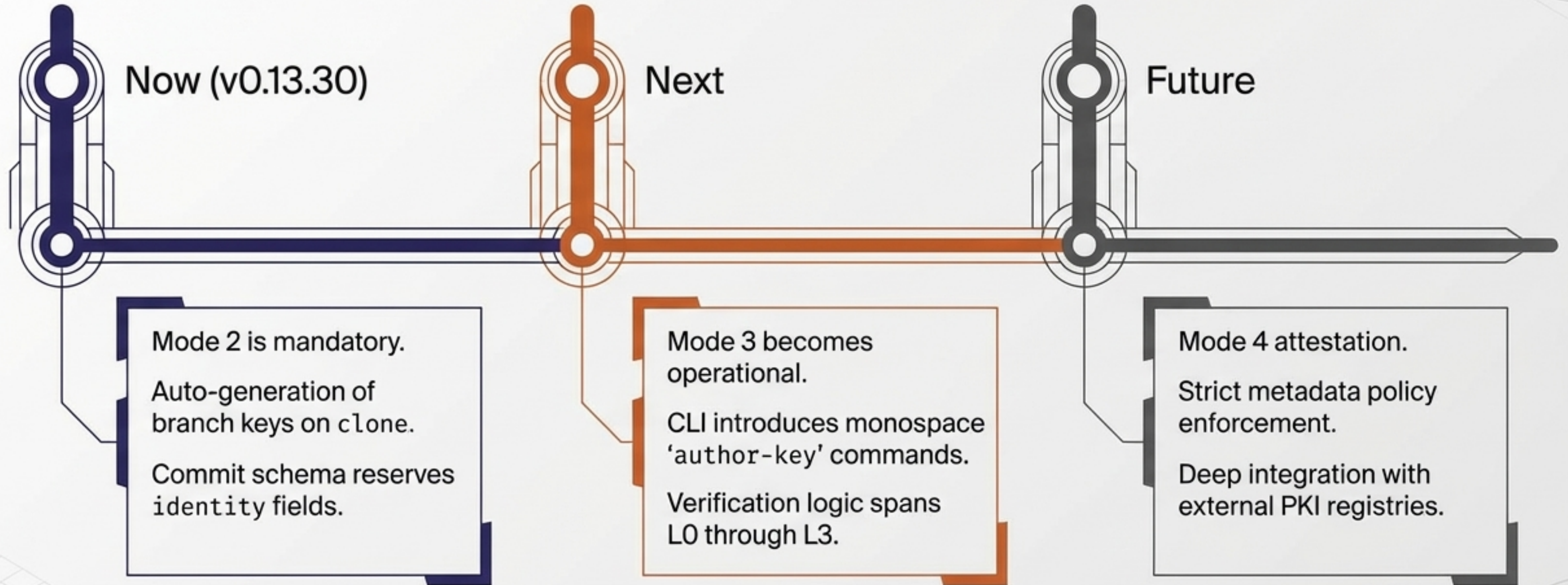
```
{  
  "commit_id": "8f9a2b...",  
  "branch_signature": "sig_39d...",  
  "payload_hash": "hash_7c...",  
  "author_key_id": null,  
  "author_signature": null,  
  "attestations": []  
}
```

The Insight: Signatures cover ALL fields.

It is cryptographically impossible to retroactively add an author signature to a Mode 2 commit, as altering the **null** field would instantly break the original device signature.

Provenance is permanently immutable at the moment of creation.

Implementation roadmap for the provenance spectrum



Anticipating the edge cases of enterprise identity

Open Question

Key Rotation

Do long-lived vaults require a cryptographic 'key succession' record to validate historical commits after a rotation?



Open Question

Revocation

How does the validation chain handle commits signed by a user key that is later compromised and revoked?



Open Question

Hardware Support

Extending the **CLI** to call external signing interfaces for TPMs and YubiKeys.



Open Question

Delegation

Can a human cryptographically **delegate** temporary **signing authority** to an AI agent?



Final Takeaway: The foundational architecture is built to support these **capabilities natively** as workflow and compliance demands evolve.